what is node js ----------

node js is not a language. Node js is async language

this is a Server run time Environment (run JavaScript on server environment)

Node js can connect with database

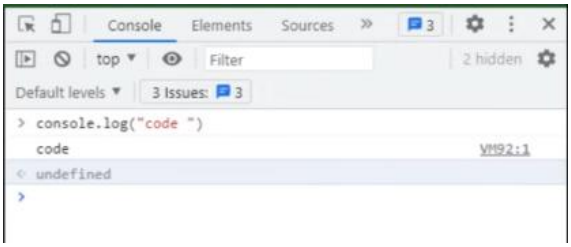
Code and Syntax is very similar to JavaScript, But not exact same

Node js free, open source

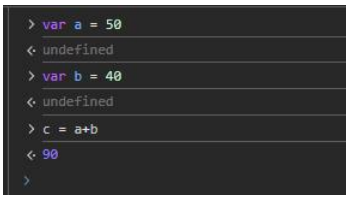
Node js use Chrome's V8 engine to execute code

Node js Version check ----->>> node -v

Npm js Version check ----->>> npm -v



With this example result shows undefined because with console.log only print the message not return anythings

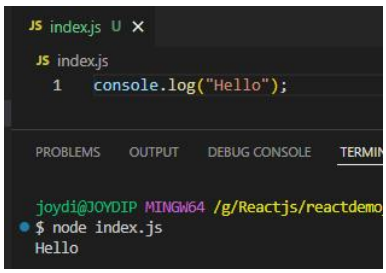


Here at the last equation result shows without undefined ***because c is return the plus value of a and b***

Run node js from folder: ----------

Create a folder.

In side folder create index.js file.



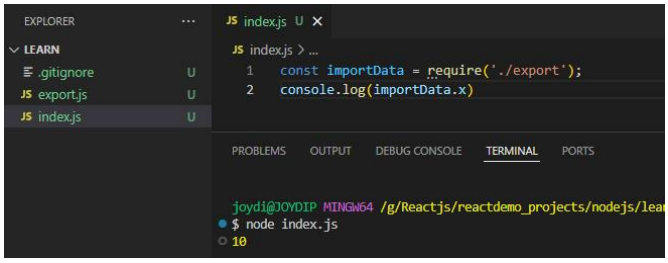
Difference between browser console and node js console ----------

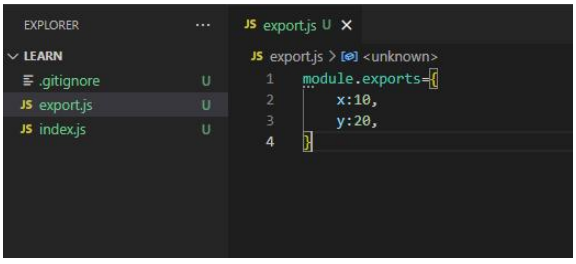
Both console are different

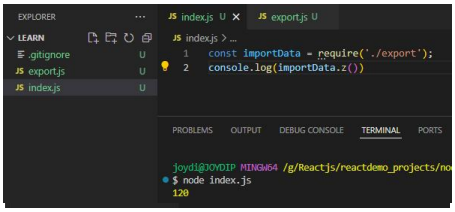
browser console is a function of browser

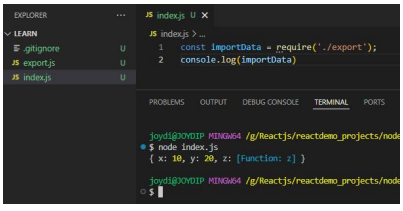
node js console is a internal module of node js because node is not run in browser

Import file in node js



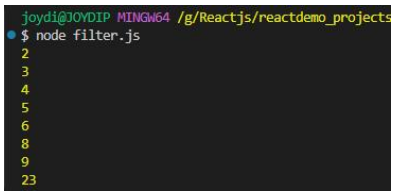


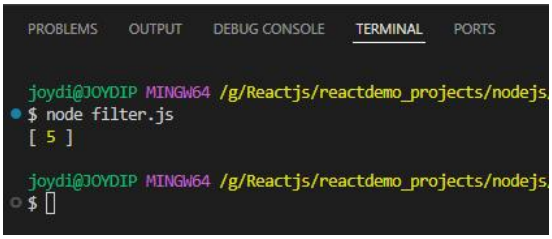


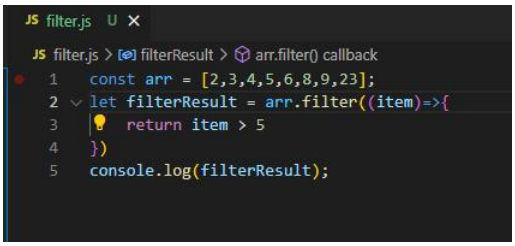


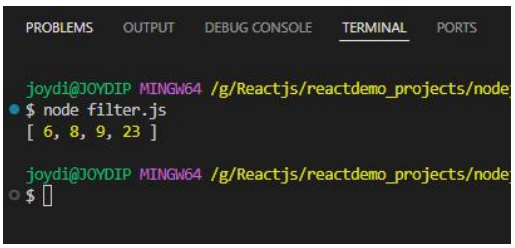
What is filter: ----------

Filter is a function which helps to find a specific data from a array. It always works with array data only.









Types of module in Node JS ----------

1. Core Module > 1.1> Global Module, 1.2> Non Global Module
2. External Module

In Node.js, **core modules** are built-in modules that come pre-packaged with the Node.js runtime. They provide essential functionality for various operations, such as working with files, streams, HTTP servers, cryptography, and more. These modules are directly available without requiring any additional installation or setup.

Characteristics of Core Modules

- **Pre-installed:** They are bundled with Node.js and don't require installation through npm.
- **Lightweight:** Optimized for performance and directly integrated with Node.js.
- **Always Available:** You can access them by simply using the `require()` function.

List of Node.js Core Modules: Other notable core modules include:

- **Buffer:** For working with binary data.
- **Stream:** For handling streaming data.
- **URL:** For URL resolution and parsing.
- **Timers:** For managing timeouts and intervals.
- **Zlib:** For compression and decompression.

Why Use Core Modules?

- No need to install additional dependencies.
- Highly efficient and optimized for Node.js.
- Reduce external dependencies in your project.

Global modules ----------

In Node.js, global modules refer to Globally Installed Modules. These are modules installed system-wide and can be accessed from any project on your machine. They are not tied to a specific project directory.
To install a module globally, use the **-g** flag with npm:

Example: npm install -g <module_name>

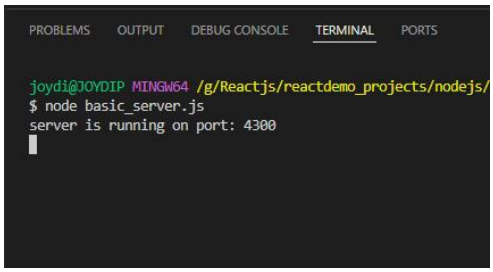
Make a basic server and output on server: ----------

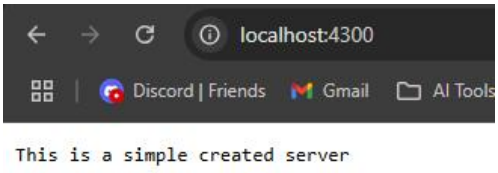
In Node.js, the **http module** is a **core module** used to create HTTP servers and clients. It provides essential functionalities to interact with the **HTTP protocol** , enabling Node.js applications to handle web requests and responses.

Key Features of the http Module

- **Core Module:** Pre-installed with Node.js, no need to install it separately.
- **Web Server Creation:** You can create an HTTP server to handle client requests.
- **HTTP Client:** Allows making HTTP requests to external servers.
- **Customizable:** Offers flexibility to define how the server handles incoming requests and sends responses.

Basic Usage of the http Module






```
const http = require('http');

const server = http.createServer((req, resp)=>{
    const url = req.url;
    if(url === '/') {
        resp.writeHead(200, {'Content-Type': 'text/plain'});
        resp.end("Home Page");
    }else if (url === '/about'){
        resp.writeHead(200, {'Content-Type': 'text/plain'});
        resp.end("About Page");
    }else{
        resp.writeHead(400, {'Content-Type': 'text/plain'});
        resp.end("Not Found !!");
    }
});

const port = 3000;
server.listen(port, ()=>{ console.log(`port is runing on ${port}`); });
```

what is package.json ----------

In Node.js, package.json is a file that provides metadata about a project or module. It serves as the central configuration file for a Node.js project and includes information about the project, its dependencies, scripts, and other settings.

Creating a package.json File

create a package.json file using the following command: npm init

```
{
  "name": "learn",
  "version": "1.0.0",
  "description": "",
  "main": "index.js",
  "scripts": {
    "test": "echo \"Error: no test specified\" && exit 1"
  },
  "author": "",
  "license": "ISC"
```

```
}
```

Customize Package.json

```
{
  "name": "learn",
  "version": "1.0.0",
  "description": "",
  "main": "index.js",
  "scripts": {
    "start": "node index.js",
    "test": "echo \"Error: no test specified\" && exit 1"
  },
  "keywords": ["nodejs", "example", "app"],
  "author": "author name",
  "license": "ISC"
}
```

```
"scripts": {
    "start": "node index.js",
    "test": "echo \"Error: no test specified\" && exit 1"
  },
```

With this above start key we can run code by command: npm start

Create a Simple API: ----------

```
const http = require("http");
http.createServer((req, resp)=>{
    resp.writeHead(200, { 'content-type' :'application/json' });
    resp.write(JSON.stringify({ "name": "nodejs", "type": "javascript" }));
    resp.end();
}).listen(5000);
```

Pretty-print ☐

```
{"name":"nodejs","type":"javascript"}
```

Get Simple API Data from other file : ----------

simple_api_data.js U

```
const data = [
    { "name": "nodejs", "type": "javascript" },
    { "name": "reactjs", "type": "javascript" },
    { "name": "angular", "type": "javascript" },
];

module.exports = data;
```

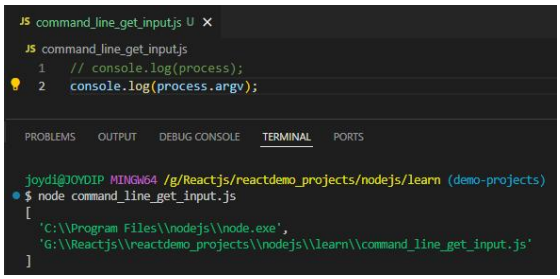
simple_api.js U

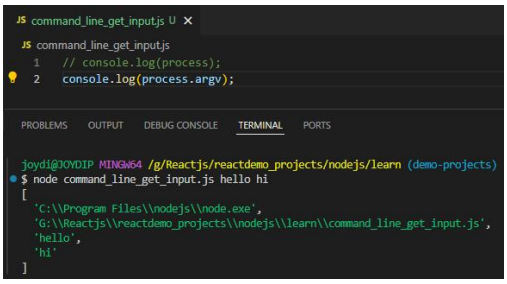
```
const http = require("http");
http.createServer((req, resp)=>{
    resp.writeHead(200, { 'content-type' :'application/json' });
    resp.write(JSON.stringify(data));
    resp.end();
}).listen(5000);
```

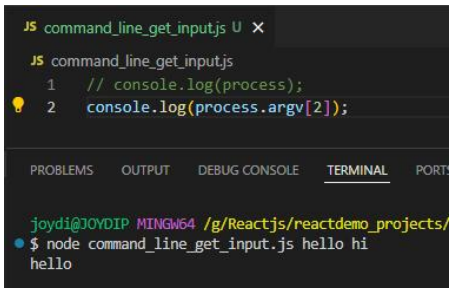
Pretty-print ☐

```
[{"name":"nodejs","type":"javascript"},{"name":"reactjs","type":"javascript"},{"name":"angular","type":"javascript"}]
```

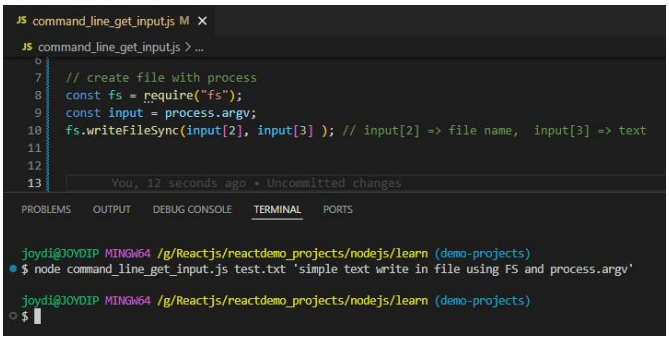
Get input from command line : ----------

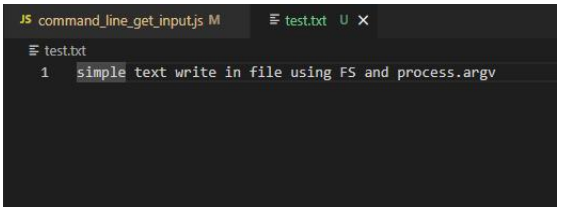






Create a file with text using fs core module and process.argv ----------





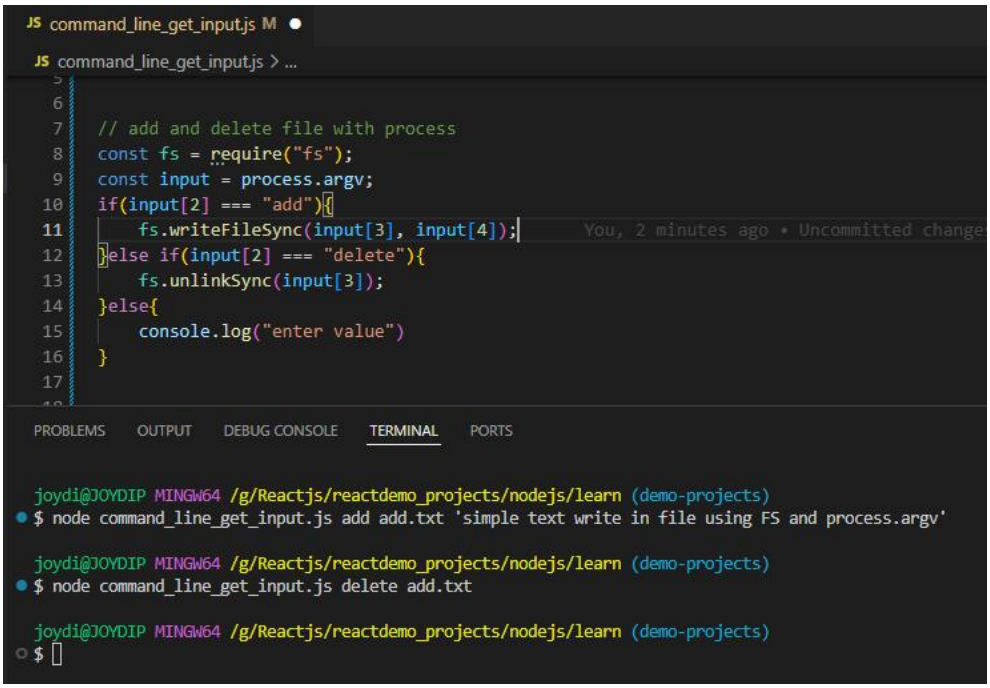
For Create a file ----------

```
fs.writeFileSync
```

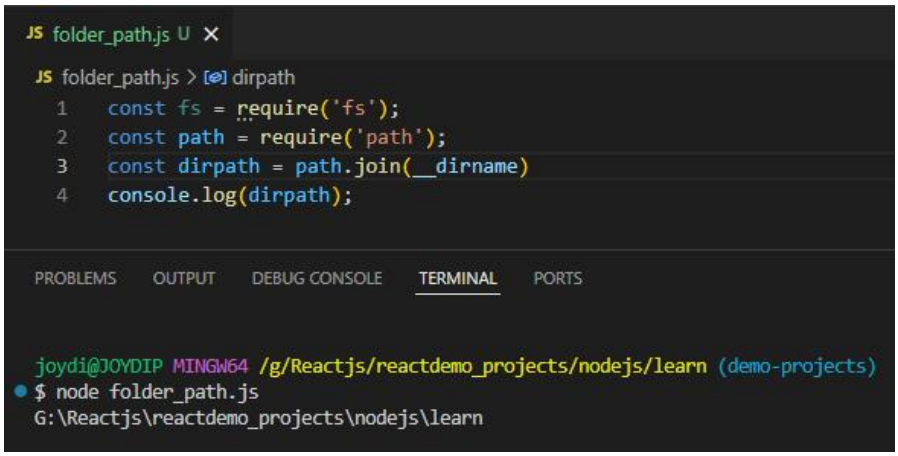
For remove a file ----------

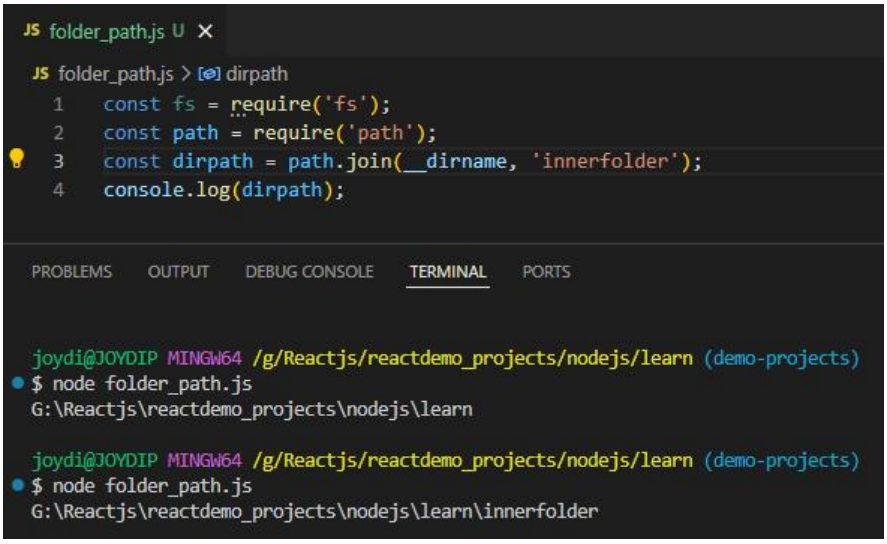
```
fs.unlinkSync('filename')
```

Add and remove file using fs module and process ----------

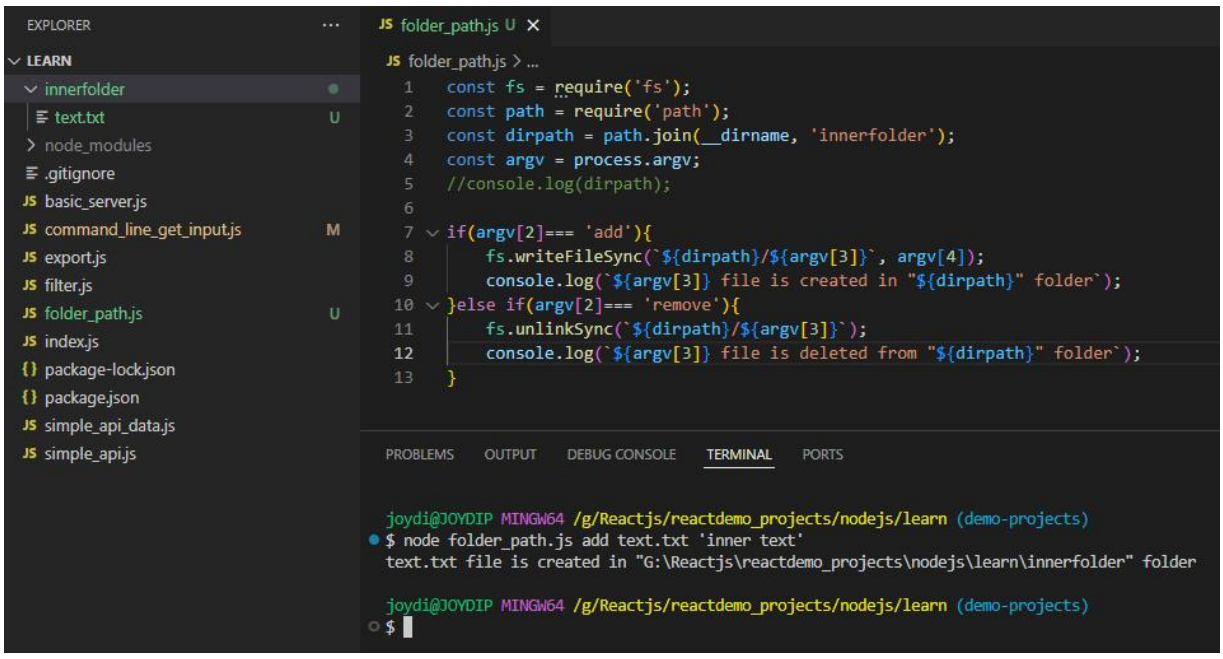


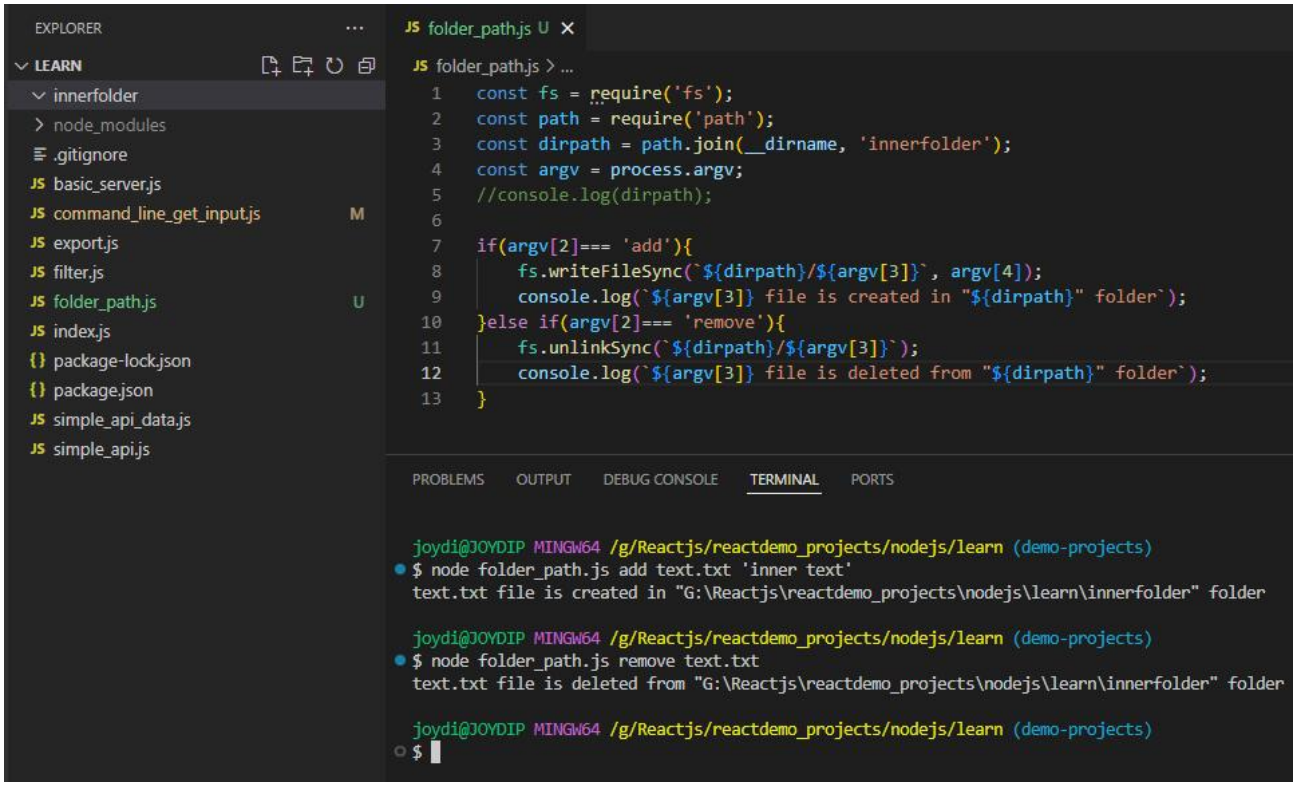
Get folder path: ----------



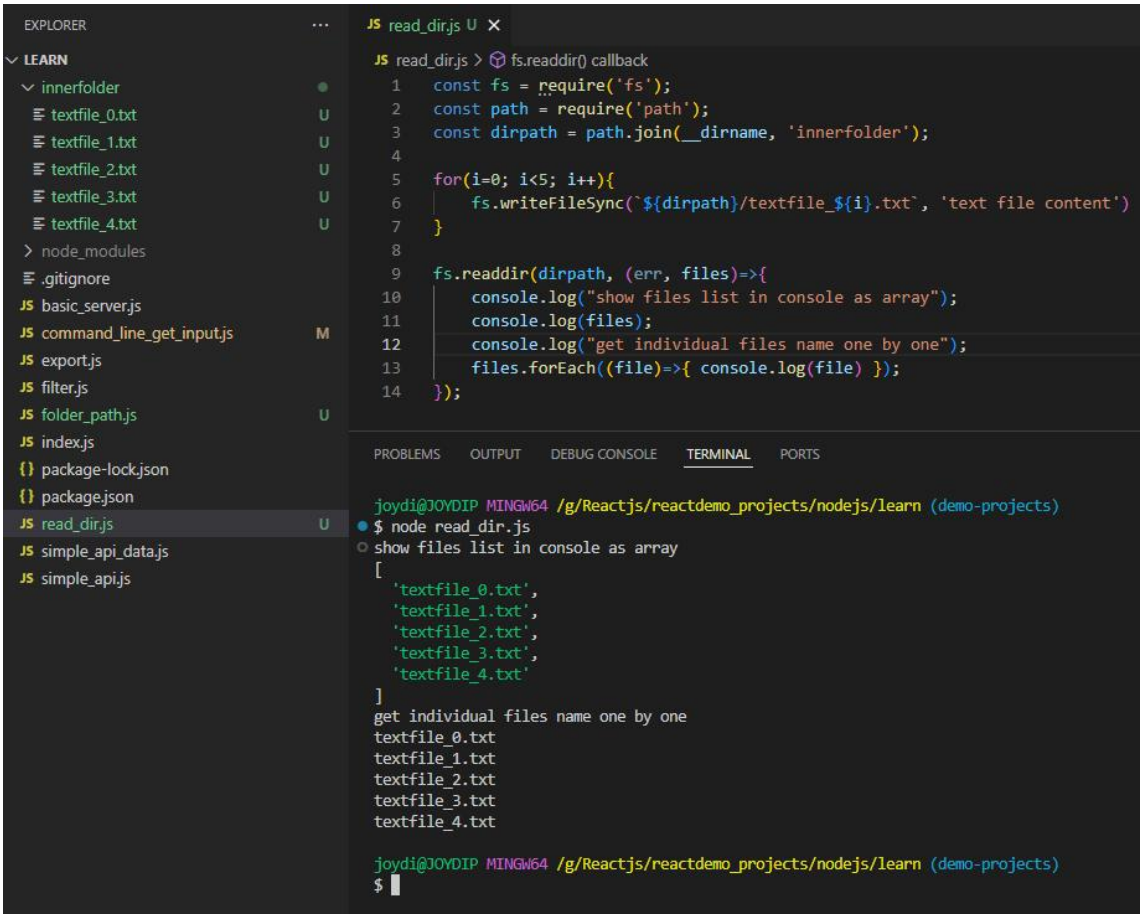


Create file in inner folder of project folder: ----------





Create and get file list from a inner folder of project folder: ----------



Get file content / edit file content / rename file : ----------

```
read_file.js U
read_file.js > filename
1   const fs = require('fs');
2   const path = require('path');
3   const dirpath = path.join(__dirname, 'innerfolder');
4   const filename = dirpath+'/textfile_0.txt'
5
6   fs.readFile(filename, (err, item)=>{
7       if(!err){
8           console.log(item);
9       }
10  })

PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS

joydi@JOYDIP MINGW64 /g/Reactjs/reactdemo_projects/nodejs/learn (demo-projects)
$ node read_file.js
<Buffer 74 65 78 74 20 66 69 6c 65 20 63 6f 6e 74 65 6e 74>

joydi@JOYDIP MINGW64 /g/Reactjs/reactdemo_projects/nodejs/learn (demo-projects)
$
```

```
read_file.js U
read_file.js > ...
1   const fs = require('fs');
2   const path = require('path');
3   const dirpath = path.join(__dirname, 'innerfolder');
4   const filename = dirpath+'/textfile_0.txt'
5
6   fs.readFile(filename, 'utf8' , (err, item)=>{
7       if(!err){
8           console.log(item);
9       }
10  })

PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS

joydi@JOYDIP MINGW64 /g/Reactjs/reactdemo_projects/nodejs/learn (demo-projects)
$ node read_file.js
<Buffer 74 65 78 74 20 66 69 6c 65 20 63 6f 6e 74 65 6e 74>

joydi@JOYDIP MINGW64 /g/Reactjs/reactdemo_projects/nodejs/learn (demo-projects)
$ node read_file.js
text file content

joydi@JOYDIP MINGW64 /g/Reactjs/reactdemo_projects/nodejs/learn (demo-projects)
$
```

```
read_file.js U   textfile_0.txt U
read_file.js > ...
1   G:\Reactjs\reactdemo_projects\nodejs\learn\read_file.js • Untracked
2   const path = require('path');
3   const dirpath = path.join(__dirname, 'innerfolder');
4   const filename = dirpath+'/textfile_0.txt'
5
6   fs.appendFile(filename, ' @@@@@ new text added 00001', (err)=>{
7       if(!err){
8           console.log("file conetnt updated");
9       }
10  })
11

PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS

joydi@JOYDIP MINGW64 /g/Reactjs/reactdemo_projects/nodejs/learn (demo-projects)
$ node read_file.js
file conetnt updated

joydi@JOYDIP MINGW64 /g/Reactjs/reactdemo_projects/nodejs/learn (demo-projects)
$
```

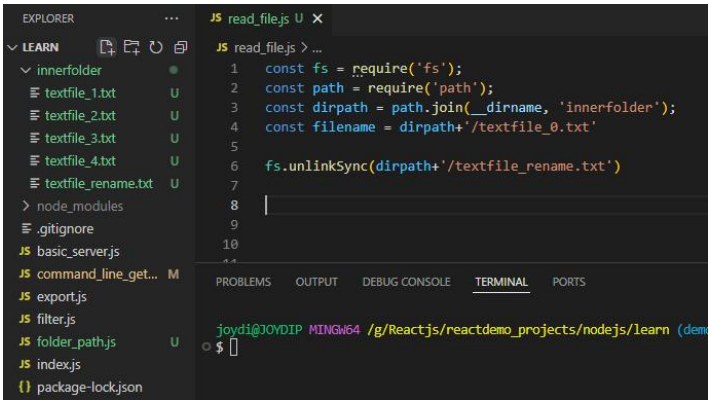
```
read_file.js U   textfile_0.txt U
innerfolder > textfile_0.txt
1   text file content @@@@@ new text added 00001

PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS

joydi@JOYDIP MINGW64 /g/Reactjs/reactdemo_projects/nodejs/lear
$ node read_file.js
file conetnt updated

joydi@JOYDIP MINGW64 /g/Reactjs/reactdemo_projects/nodejs/lear
$
```

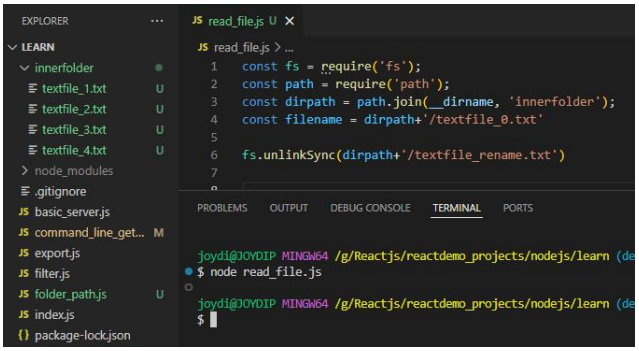
```
EXPLORER
LEARN
  innerfolder
    textfile_0.txt U
    textfile_1.txt U
    textfile_2.txt U
    textfile_3.txt U
    textfile_4.txt U
  node_modules
  .gitignore
  basic_server.js
  command_line_get... M
  export.js
  filter.js
  folder_path.js U
  index.js
  package-lock.json
  package.json
  read_dir.js U
read_file.js U
read_file.js > ...
1   const fs = require('fs');
2   const path = require('path');
3   const dirpath = path.join(__dirname, 'innerfolder');
4   const filename = dirpath+'/textfile_0.txt'
5
6   // rename file name
7   fs.rename(filename, dirpath+'/textfile_rename.txt', (err)=>{
8       if(!err){
9           console.log("file name updated");
10      }
11  })
12
13

PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS

joydi@JOYDIP MINGW64 /g/Reactjs/reactdemo_projects/nodejs/learn (demo-projects)
$
```

```
EXPLORER
LEARN
  innerfolder
    textfile_1.txt U
    textfile_2.txt U
    textfile_3.txt U
    textfile_4.txt U
    textfile_rename.txt U
  node_modules
  .gitignore
  basic_server.js
  command_line_get... M
  export.js
  filter.js
  folder_path.js U
  index.js
  package-lock.json
  package.json
  read_dir.js U
  read_file.js U
  simple_api_data.js
read_file.js U
read_file.js > ...
1   const fs = require('fs');
2   const path = require('path');
3   const dirpath = path.join(__dirname, 'innerfolder');
4   const filename = dirpath+'/textfile_0.txt'
5
6   // rename file name
7   fs.rename(filename, dirpath+'/textfile_rename.txt', (err)=>{
8       if(!err){
9           console.log("file name updated");
10      }
11  })
12
13

PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS

joydi@JOYDIP MINGW64 /g/Reactjs/reactdemo_projects/nodejs/learn (demo-proje
$ node read_file.js
file name updated

joydi@JOYDIP MINGW64 /g/Reactjs/reactdemo_projects/nodejs/learn (demo-proje
$
```

Buffer: **temporary file location for save data using file system to store data node require some memory**

Delete file : ----------






```
const path = require('path');

//get current folder path
const currentFolderPath = path.dirname(__filename);
console.log("current folder path with folder name: ", currentFolderPath);
//result: current folder path with folder name:  G:\Reactjs\reactdemo_projects\nodejs\learn

// get folder name
const currentFolderName = path.basename(__dirname);
console.log("current folder Name: ", currentFolderName);
//result: current folder Name:  learn

// get file name
const currentFileName = path.basename(__filename);
console.log("current file Name: ", currentFileName);
//result: current file Name:  path.js

// get file extension
const currentFileExtension = path.extname(__filename);
console.log("current file extension: ", currentFileExtension);
//result: current file extension: .js

// join path
const joinpath = path.join("/user", "data", "projects");
console.log("Join Path : ", currentFolderPath+joinpath);
//result: Join Path :  G:\Reactjs\reactdemo_projects\nodejs\learn\user\data\projects
```

```
const fs = require('fs');
const path = require('path');

// create path for Data Folder
const datafolder = path.join(__dirname, 'data');

// checking that Data folder is there or not, if not then create a data folder
if(!fs.existsSync(datafolder)){
    fs.mkdirSync(datafolder);
    console.log('data folder created');
}

// create a file inside data folder and add some content in the file
const filePath = path.join(datafolder, "data01.txt");
fs.writeFileSync(filePath, "file text content added");

// read conetnt from the created file
const readFileContent = fs.readFileSync(filePath, 'utf8');
console.log("File Content: ", readFileContent);
```

```
// appended some new text inside the file
fs.appendFileSync(filePath, "\nNew text Content appended");
const readAppendedFileContent = fs.readFileSync(filePath, 'utf8');
console.log("Appended Content: ", readAppendedFileContent);

// create file with Async way-------------------------------
const AsyncFilePath = path.join(datafolder, 'async-file.txt');
fs.writeFile(AsyncFilePath, 'Text content for Async file', (err)=>{
    if(err) throw err;
    console.log("Async file is created");

    // read async file
    fs.readFile(AsyncFilePath, 'utf8',(err, data)=>{
        if(err) throw err; console.log("Async File content: ", data);

        // append new content
        fs.appendFile(AsyncFilePath, "\nNew Async content append", (err)=>{
            if(err) throw err;
            console.log("New text line is added in async file");
            fs.readFile(AsyncFilePath, 'utf8', (err, updatedData)=>{
                if(err) throw err;
                console.log("Appended content of Async file: ", updatedData);
            })
        })
    })
})
```

Callback Function ----------

A callback function is a function passed as an argument to another function, which can be ***invoked*** (or "called back") ***after the completion of a certain task or operation***. This allows ***asynchronous code execution***, where certain actions can happen after other operations

```
function person(name, callback){
    console.log(`Hello, ${name}`);
    callback();
}

function country(){
    console.log("India");
}

person("Rahul Roy", country);
```

Result:
Hello, Rahul Roy
India

Callback Hell ----------

Callback Hell refers to the situation where multiple nested callback functions are used, leading to code that is difficult to read, maintain, and debug. ***It typically occurs in scenarios where asynchronous operations are performed in sequence***, ***each depending on the result of the previous one***. The more the nested callbacks, the harder it becomes to understand the flow of the code.

```
const fs = require('fs');

fs.readFile('file1.txt', 'utf8', (err, data1) => {
    if (err) return console.error(err);
    fs.readFile('file2.txt', 'utf8', (err, data2) => {
        if (err) return console.error(err);
        fs.readFile('file3.txt', 'utf8', (err, data3) => {
            if (err) return console.error(err);
            console.log(data1, data2, data3);
```

```
        });
    });
});
```

Problems in This Example:

- ***Nested Structure:*** Each ***fs.readFile*** call is nested within the callback of the previous call.
- ***Error Handling:*** Each level of nesting requires its own error handling.
- ***Readability:*** The deeper the nesting, the harder it is to follow the code flow.

Promise ----------

```
const promiseObject = new Promise((resolve, reject)=>{
    let x = 10;
    if(x < 5){
        resolve("success and resolved");
    }else{
        reject("failed and rejected");
    }
});

promiseObject.then((message)=>{
    console.log("Resolved Message: ", message);
}).catch((message)=>{
    console.log("error: ", message);
})
```

async/await ----------

Async is exactly what promises are doing to handle asynchronous operations, But we use that because `async/await` is a syntactic sugar over Promises, providing a way to write asynchronous code that is more readable, maintainable, and easier to handle compared to traditional promise chaining

```
        function delayFn(time){
            return new Promise((resolve)=>setTimeout(resolve, time))
        }

        async function dalayedGreet(name){
            await delayFn(2000)
            console.log(name);
        }

        dalayedGreet("Raja Roy");
```

async/await Error handling ----------

```
async function division(num1, num2) {
    try{
        if(num2 === 0) throw new Error('Can not divide by 0');
        return num1/num2

    }catch(error){
        console.log("error --- ", error);
        return null
    }
}

async function mainFu() {
    console.log(await division(10,2))
    console.log(await division(10,0))
}

mainFu()
```

Result:

```
$ node async-await.js
5
error ---  Error: Can not divide by 0
   at division (G:\Reactjs\reactdemo_projects\nodejs\learn\async-await.js:15:30)
   at mainFu (G:\Reactjs\reactdemo_projects\nodejs\learn\async-await.js:26:23)
null
```

EventEmitter ----------

EventEmitter is a core module in Node.js that allows you to create and handle custom events. It is part of the events module and provides a mechanism to emit (trigger) events and handle them with listeners (event handlers). This pattern is widely used in Node.js for handling asynchronous events.

```
const EventEmitter = require('events');
const myEmitter = new EventEmitter();

// Register an event listener for 'greet' event
myEmitter.on('greet', (name) => {
    console.log(`Hello, ${name}!`);
});

// Emit the 'greet' event
myEmitter.emit('greet', 'Alice');
```

Result:

```
$ node event-emitter.js
Hello, Alice!
```

Custom Event Listener ----------

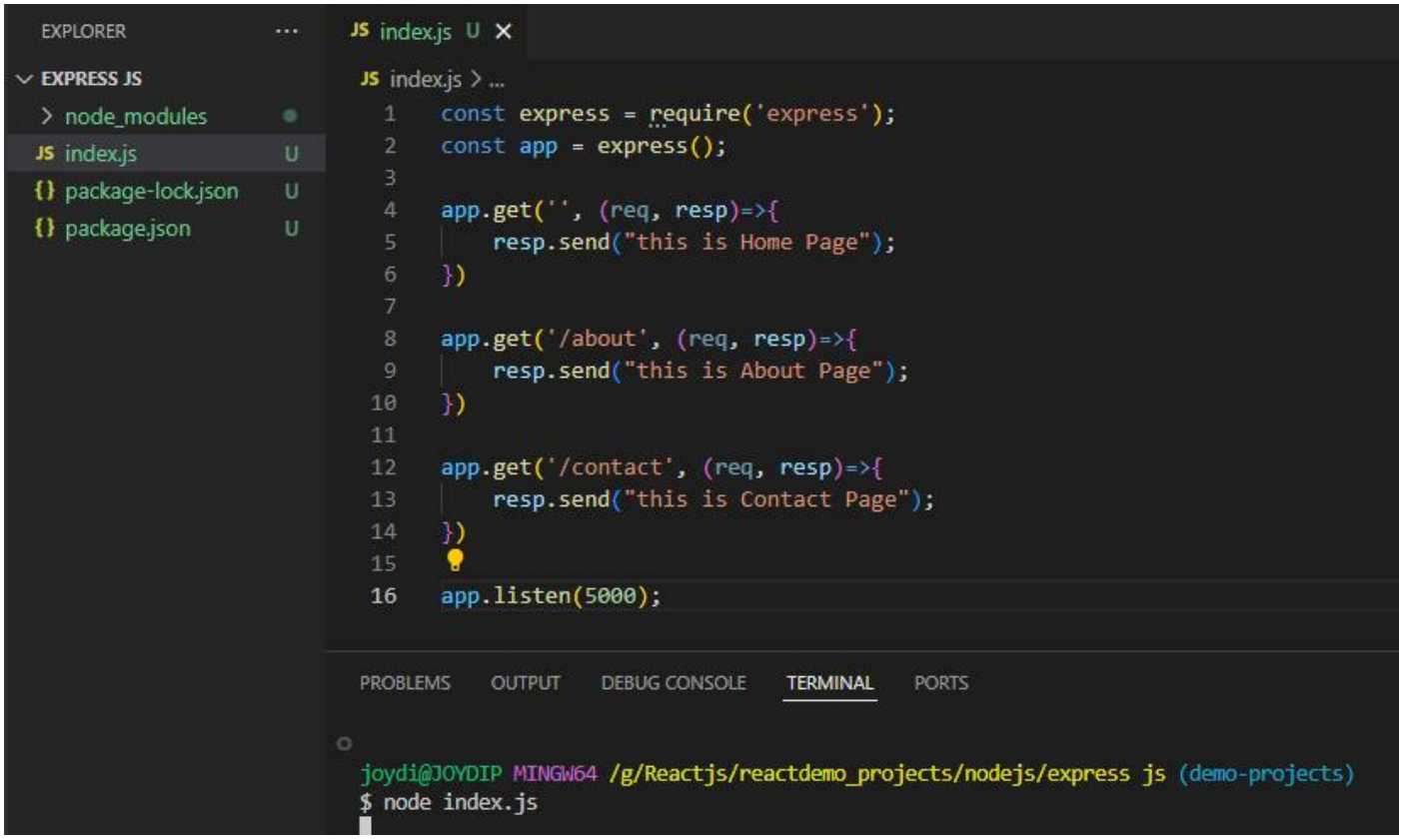
```
const EventEmitter = require('events');
class MyCustomEmitter extends EventEmitter{
    constructor(){
        super();
        this.greeting = "Hello"
    }

    greet(name){
        this.emit('greeting', `${this.greeting} ${name}` )
    }
}

const myCustomEmitter = new MyCustomEmitter();

myCustomEmitter.on('greeting', (input)=>{
    console.log("greeting event", input);
});

myCustomEmitter.greet("Alice");
```

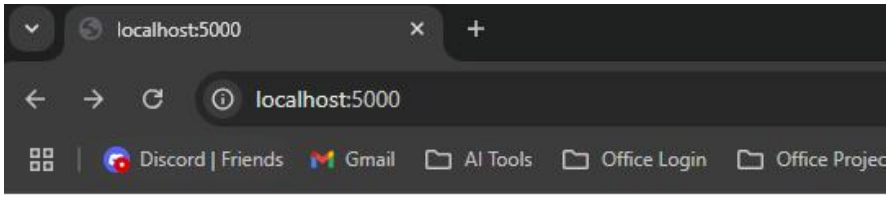
Result:

```
$ node event-emitter.js
greeting event Hello Alice
```

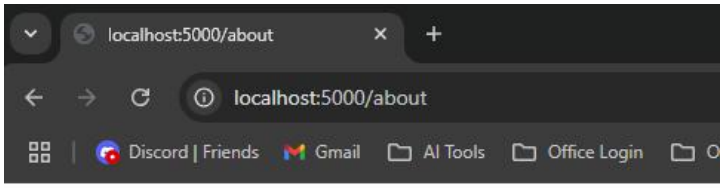
---------- Express JS ----------

Create simple pages using Express ----------





this is Home Page



this is About Page

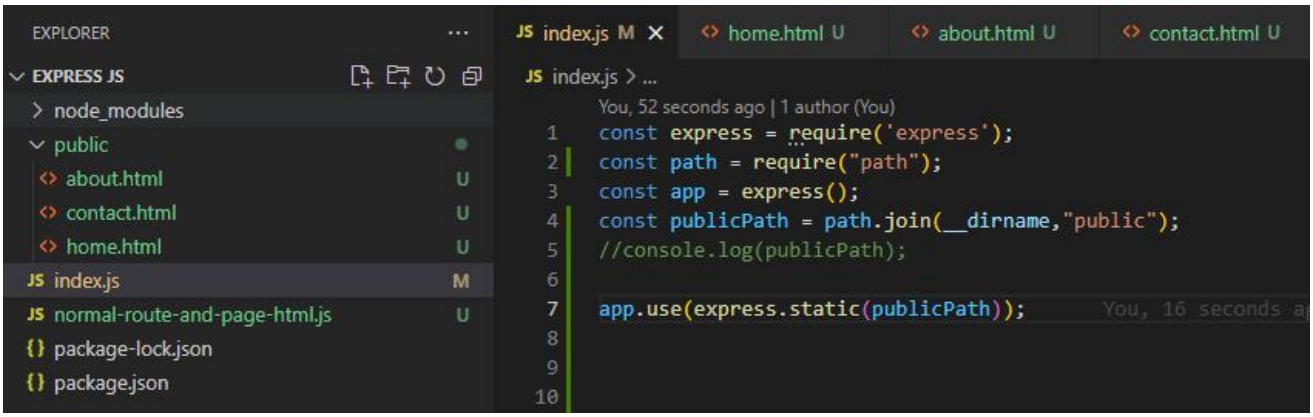
Create simple and dynamic routes using Express ----------

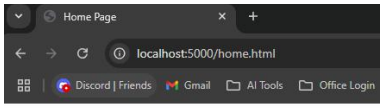
```
const express = require('express');
const app = express();

app.get('', (req, resp)=>{
    resp.send("<h2>this is Home Page</h2>");
})

app.get('/about', (req, resp)=>{
    resp.send("<h2>this is About Page</h2>");
})

app.get('/contact', (req, resp)=>{
    resp.send("<h2>this is Contact Page</h2>");
})

app.get('/products', (req,resp)=>{
    const products = [
        {id:1, name: "product 1"}, {id:2, name: "product 2"}, {id:3, name: "product 3"},
    ]
    resp.json(products);
})

// get single value
app.get('/product/:id', (req,resp)=>{
    const productID = parseInt(req.params.id)
    const products = [
        {id:1, name: "product 1"}, {id:2, name: "product 2"}, {id:3, name: "product 3"},
    ]
    const singleProduct = products.find((product)=> { return(product.id === productID) })
    if(singleProduct){
        resp.json(singleProduct);
    }else{
        resp.status(404).send("product not found")
    }
})

app.listen(5000);
```

Result: http://localhost:5000/product/1{"id":1,"name":"product 1"}

Load static HTML files from public folder ----------

static method use for load static html pages from public folder





Home Page

Remove extension from URL ----------

```
const express = require('express');
const path = require("path");
const app = express();
const publicPath = path.join(__dirname,"public");

app.get('', (req,resp)=>{
    resp.sendFile(`${publicPath}/home.html`);
});

app.get('/aboutus', (req,resp)=>{
    resp.sendFile(`${publicPath}/about.html`);
});

app.get('/contact-us', (req,resp)=>{
    resp.sendFile(`${publicPath}/contact.html`);
});

app.get('*', (req,resp)=>{
    resp.sendFile(`${publicPath}/404.html`);
});


app.listen(5000);
```

Template Engine ----------

A template engine in web development is a tool that allows you to ***generate dynamic HTML page***s by combining static templates with data.

Why Use a Template Engine?

- Generate HTML pages dynamically based on data
- Keep HTML templates separate from application logic, promoting clean code.
- Provides syntax for loops, conditionals, and variable interpolation directly within the HTML.

Popular Template Engines in Node.js

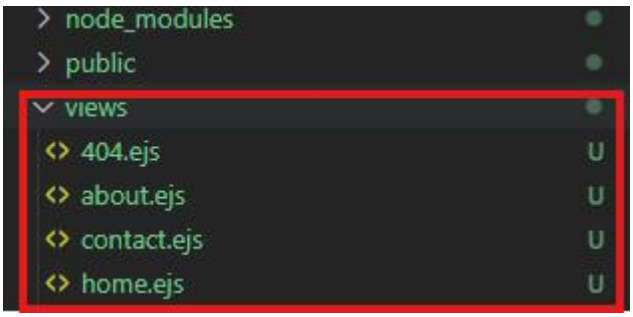
EJS (Embedded JavaScript)
Pug (formerly Jade)
Handlebars
Mustache

How to use ?

```
const express = require('express');
const path = require("path");
const app = express();
const publicPath = path.join(__dirname,"public");

// Set EJS as the template engine
app.set('view engine','ejs')

app.get('', (req,resp)=>{
    //resp.sendFile(`${publicPath}/home.html`);
    resp.render('home');
});

app.get('/aboutus', (req,resp)=>{
    resp.render('about');
});

app.get('/contact-us', (req,resp)=>{
    resp.render('contact');
});

app.get('*', (req,resp)=>{
```

```
    resp.render('404');
});

app.listen(5000);
```

File Structure ----------



Send data to ejs template page ----------

```
const express = require('express');
const path = require("path");
const app = express();
const publicPath = path.join(__dirname,"public");

// Set EJS as the template engine
app.set('view engine','ejs')
app.get('', (req,resp)=>{
    const data = {
        name: 'Rahul Roy',
        email: 'rahul@gmail.com'
    }
    resp.render('home', {data});
});

app.get('*', (req,resp)=>{
    resp.render('404');
});
```

home.ejs ----------

```
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>Home EJS Page</title>
</head>
<body>

    <h1>Home EJS Page</h1>
    <h2> Welcome, <%= data.name %> </h2>
</body>
</html>
```

Loop in ejs ----------

```
app.get('', (req,resp)=>{
    const data = {
        name: 'Rahul Roy',
        email: 'rahul@gmail.com',
        skills: ['html', 'css', 'javascript', 'node js']
    }
    resp.render('home', {data});
});
```

home.ejs ----------

```
<ul>
    <% data.skills.forEach((item)=>{ %>
        <li><%= item %></li>
    <% }) %>
</ul>
```

Common header in ejs ----------

```
<%- include("common/header")  %>
<h1>Home EJS Page</h1>
<h2> Welcome, <%= data.name %> </h2>
<ul>
    <% data.skills.forEach((item)=>{ %>
        <li><%= item %></li>
    <% }) %>
</ul>
```

*** For include HTML we need to use** `<%- include("filename")  %>`

**** For show JavaScript / data we need to use** `<li><%= item %></li>`

---------- Middleware ----------

In web development, middleware refers to functions that ***sit between the request and the response*** in a server's request-response cycle. Middleware functions are commonly used in frameworks like ***Express.js to handle tasks such as logging, authentication, parsing, routing***, and ***error handling***.

We can ***access*** and ***modify request and the response*** in ***routes***

What Middleware Does

- **Intercepts Requests:** Middleware can inspect, modify, or terminate requests before they reach the final handler.
- **Processes Responses:** Middleware can modify or terminate responses before they are sent to the client.
- **Chains Execution:** Middleware functions can pass control to the next middleware in the stack using the next() function.

Middleware in Express.js

In Express.js, middleware functions are functions that have access to:

- **Request** (req): The incoming HTTP request object.
- **Response** (res): The outgoing HTTP response object.
- **Next** (next): A function to pass control to the next middleware.

Create Middleware ----------

```
// create middleware
const routeMiddleware = (req, resp, next) => {
    console.log("middleware start working");
}

// use middleware
app.use(routeMiddleware);
```

Now if we run the index.js file we will see that page is continuously loading. So for that we need to add next() for goto further process like below and we will see in browser that page now loaded properly

```
        // create middleware
        const routeMiddleware = (req, resp, next) => {
            console.log("middleware start working");
            next();
        }

        // use middleware
        app.use(routeMiddleware);
```

Middleware Example ----------

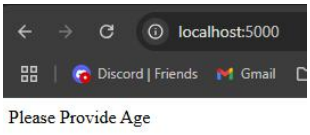
```
const express = require("express");
const app = express();

const loggerMiddleware = (req,resp,next) =>{
    const timestamp = new Date().toISOString();
    console.log(`${timestamp} from ${req.method} to ${req.url}`);
    next();
}
app.use(loggerMiddleware);
app.get('', (req, resp)=>{
    resp.send("<h2>this is Home Page</h2>");
})

app.get('/about', (req, resp)=>{
    resp.send("<h2>this is About Page</h2>");
})

app.listen(5000);
```

Result: visit the all page from browser and then check console
$ node simple-middleware.js
2025-01-11T11:19:57.195Z from GET to /about
2025-01-11T11:20:20.512Z from GET to /

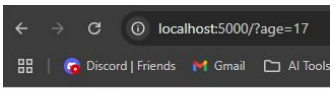
Middleware Example (Application Level Middleware) ----------

```
// create middleware
const routeMiddleware = (req, resp, next) => {
    console.log("middleware start working");
    if(!req.query.age){
        resp.send("Please Provide Age")
    }else if(req.query.age < 18){
        resp.send("You are not eligible")
    }else{
        next();
    }
}
```

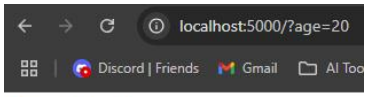
Middleware Example view ----------



Please Provide Age



You are not eligible



Common Header

Home EJS Page

Welcome, Rahul Roy

Types of Middleware ----------

Type	Scope	Examples
Application-Level	Global	Logging, Authentication
Router-Level	Specific routes	`/api` middleware
Built-in	Provided by Express.js	`express.json()`, `express.static()`
Third-Party	External packages	`morgan`, `helmet`, `cors`
Error-Handling	Handles errors	Error logging
Request-Specific	Per route/method	Route-specific logic
Custom	Developer-defined	Authentication, Validation
Static	Serves static files	`express.static()`
Conditional	Executes based on conditions	API key check
Streaming	Processes file uploads	`multer`, `busboy`
Debugging/Monitoring	Debugging/Profiling	`debug`, `express-status-monitor`

Router Level Middleware Example ----------

```
const express = require('express');
const app = express();
const path = require("path");
const publicPath = path.join(__dirname,"public");

// Set EJS as the template engine
app.set('view engine','ejs')

// create middleware
const routeMiddleware = (req, resp, next) => {
    console.log("middleware start working");
    if(!req.query.age){
        resp.send("<h2>Please Provide Age</h2>")
    }else if(req.query.age < 18){
        resp.send("<h2>You are not eligible</h2>")
    }else{
        next();
    }
}
app.get('', (req,resp)=>{
    const data = {
        name: 'Rahul Roy',
        email: 'rahul@gmail.com',
        skills: ['html', 'css', 'javascript', 'node js']
    }
    resp.render('home', {data});
});

app.get('/aboutus', routeMiddleware, (req,resp)=>{
    resp.render('about');
});

app.listen(5000);
```

With the above example we can not access **Aboutus page** without age validation

How to use Middleware from separate file ----------

middleware.js

```
// create middleware
module.exports = routeMiddleware = (req, resp, next) => {
    console.log("middleware start working");

    if(!req.query.age){
        resp.send("<h2>Please Provide Age</h2>")
    }else if(req.query.age < 18){
        resp.send("<h2>You are not eligible</h2>")
    }else{
        next();
    }
}
```

index.js

```
const express = require('express');
const app = express();
const routeMiddleware = require('./middleware')
const path = require("path");
const publicPath = path.join(__dirname,"public");

// Set EJS as the template engine
app.set('view engine','ejs')

app.get('/aboutus', routeMiddleware, (req,resp)=>{
    resp.render('about');
});
```

Router Level Middleware type 2 example ----------

middleware.js

```
// create middleware
module.exports = routeMiddleware = (req, resp, next) => {
    console.log("middleware start working");

    if(!req.query.age){
        resp.send("<h2>Please Provide Age</h2>")
    }else if(req.query.age < 18){
        resp.send("<h2>You are not eligible</h2>")
    }else{
        next();
    }
}
```

index.js

```
const express = require('express');
const app = express();
const routeMiddleware = require('./middleware')
const path = require("path");
const publicPath = path.join(__dirname,"public");
const route = express.Router();


// Set EJS as the template engine
app.set('view engine','ejs')

route.use(routeMiddleware);

route.get('', (req,resp)=>{
    const data = {
        name: 'Rahul Roy',
```

```
        email: 'rahul@gmail.com',
        skills: ['html', 'css', 'javascript', 'node js']
    }
    resp.render('home', {data});
});

route.get('/aboutus', (req,resp)=>{
    resp.render('about');
});

app.get('/contact', (req,resp)=>{
    resp.render('contact');
});

app.use('/', route);

app.listen(5000);
```

With the above example we can not access **Aboutus page** and **Home page** without age validation

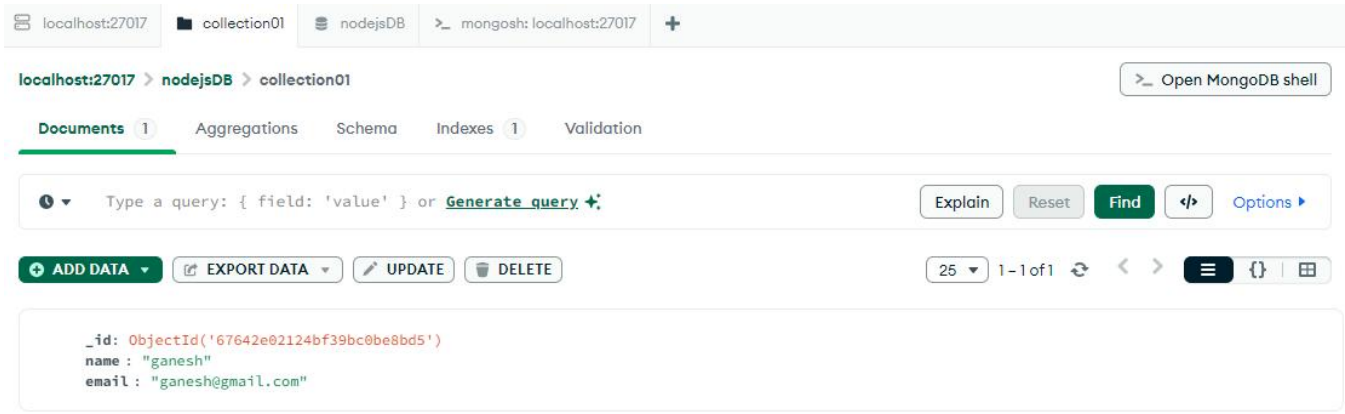
---------- MongoDB ----------

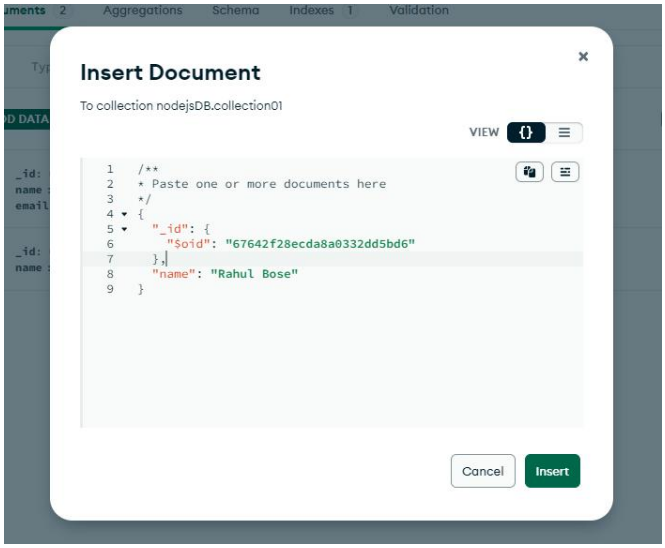
mongoDB version check: mongod --version

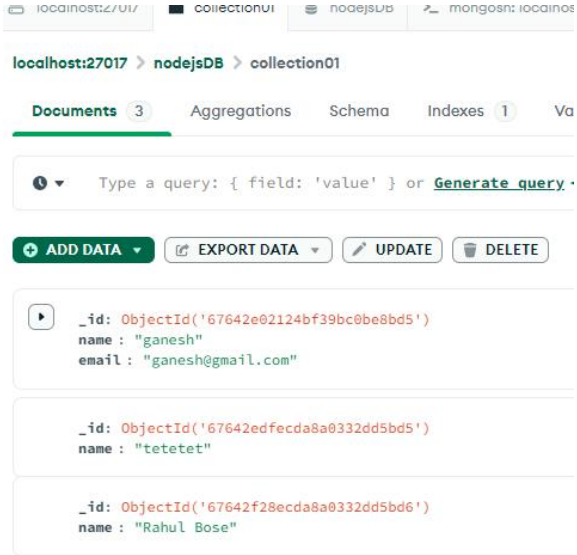
Add data in a collection using CLI:

```
db.collection01.insertOne({name:'ganesh', email:'ganesh@gmail.com'})
```

```
> db.collection01.insertOne({name:'ganesh', email:'ganesh@gmail.com'})
< {
    acknowledged: true,
    insertedId: ObjectId('67642e02124bf39bc0be8bd5')
  }
```







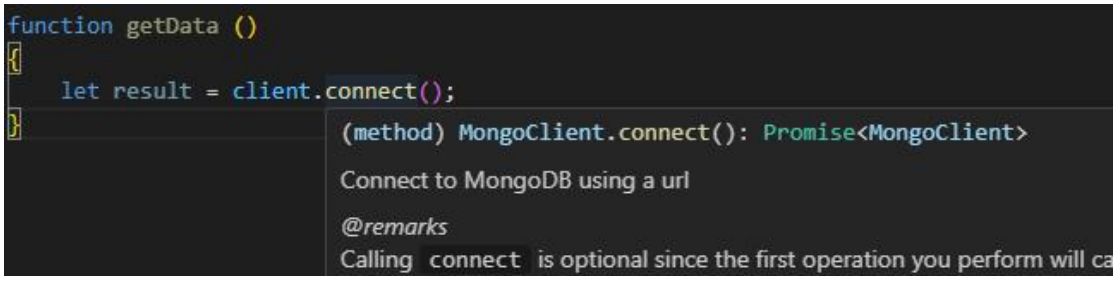
Check inserted data using CLI:

```
db.collection01.find()
```

Clear CLI: ctrl + l

----------Connect MongoDB with Node JS ----------

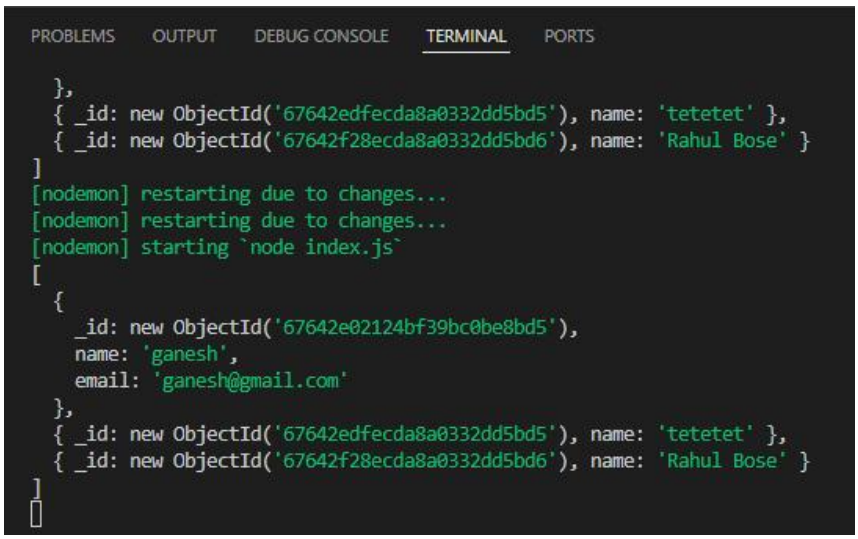
```
const {MongoClient} = require('mongodb');
const mongourl = 'mongodb://localhost:27017';
const client= new MongoClient(mongourl);

async function getData ()
{
    // connect with mongodb
    let result = await client.connect();
    // connect with database
    let db = result.db('nodejsDB');
    // select collection
    let collection = db.collection('collection01');
    // handle promise
    let response = await collection.find({}).toArray();
    console.log(response);
}
getData();
```



`client.connect()` always response a promise. Promise use for handle the delay of data fetching

For resolve this we will use **async** and **await**

```
async function getData ()
{
    let result = await client.connect();
}
```



Create config file for mongoDB connection ----------

mongoDB.js

```
const {MongoClient} = require('mongodb');
const mongourl = 'mongodb://localhost:27017';
const client= new MongoClient(mongourl);

async function dbConnect ()
{
    // connect with mongodb
    let result = await client.connect();
    // connect with database
    let db = result.db('nodejsDB');
    // select collection
    return db.collection('collection01');
    // handle promise
}

module.exports = dbConnect
```

index.js

Handle Promise to fetch data from collection type 01 ----------

```
// fetch data with promise type 01 ----------------------------------
// dbConnect() return a promise. So we use then function
dbConnect().then((resp)=>{
    // toArray() also return a promise. So we use then function again
    //console.log(resp.find().toArray())
    resp.find().toArray().then((data)=>{
        console.log(data);
    });
})
```

Handle Promise to fetch data from collection type 02 ----------

```
// fetch data with promise type 02 ----------------------------------
const fetchData = async () => {
    //console.log("fetchData started");
    let data = await dbConnect();
    //console.log(data.find().toArray())
    data = await data.find().toArray(); // return a promise
    console.log(data)
}

fetchData()
```

--------Insert Data into MongoDB using Node JS --------

```
const dbConnect = require('./mongoDB');


const insert = async () => {
    console.log("insert function working");
    // db connection
    const db = await dbConnect();
     console.log(db);

    // insert one data
    // const insertData = await db.insertOne(
    //     { name: "Ashim", email: "ashim@gmail.com" }
     // );

    // insert many data
    const insertData = await db.insertMany([
        { name: "Sudip", email: "sudip@gmail.com" },
        { name: "Ashim", email: "ashim@gmail.com" }
     ]);

    // console.log(insertData);
    if(insertData.acknowledged){
        console.log("data inserted");
    }
}

insert();
```

--------Update Data into MongoDB using Node JS --------

```
const dbConnect = require('./mongoDB');


const update = async () => {
    console.log("update function working");
    // db connection
    const db = await dbConnect();
    console.log(db);
    let updateData = await db.updateOne(
        {name: "Jubin"},
        { $set:{ name: "Jubin 2" } }
    )
    if(updateData.acknowledged){
        console.log(updateData)
    }
}

update();
```

-------- MongoDB connection pattern 2 with insert data with static data --------

mongoDB.js

```
const {MongoClient} = require("mongodb");
const mongodburl = "mongodb://localhost:27017/";
const client = new MongoClient(mongodburl);

async function dbconnection (){
    let connection = await client.connect();
    //console.log(connection);
    return connection.db("nodejsDB");
}
//dbconnection();

async function productCollection (){
    let connection = await dbconnection();
    //console.log(connection);
    return connection.collection("collection01");
}
// productCollection();

async function fetchProductData(){
    let collection = await productCollection()
    //console.log(collection)
    let data = await collection.find({}).toArray();
    console.log(data);
}
// fetchProductData();

module.exports = dbconnection
module.exports = productCollection
```

index.js

```
const productCollection = require('./mongoDB');

const insert = async () => {
    console.log("insert");
    const collection = await productCollection();
     console.log(collection);

    const insertdata = await collection.insertOne(
        { name: "eeeeeee33333", email: "eeeeeee@gmail.com" }
     );

    //console.log(insertdata);
    if(insertdata.acknowledged){
        console.log("data inserted")
    }
}

insert()

const update = async() => {
       console.log("update working");
       let collection = await productCollection();
       //console.log(collection);

       let update = await collection.updateOne(
              {name: "eeeeeee33333"},
              {$set:{ name: "wwwwwww33333"}}
       );
```

```
        if(update.acknowledged){
                console.log("data update successfully");
        }
}

update();

const deleteData = async () => {
        console.log("detele working");
        let collection = await productCollection();
        const deleteData = await collection.deleteOne({ name: "wwwwwww33333" });
        console.log(deleteData);

        if(deleteData.acknowledged && deleteData.deletedCount == 1){
                console.log("data delete successfully");
        }else{
                console.log("data not found");
        }
}

deleteData();
```

-------- MongoDB connection pattern 2 with insert data using postman --------

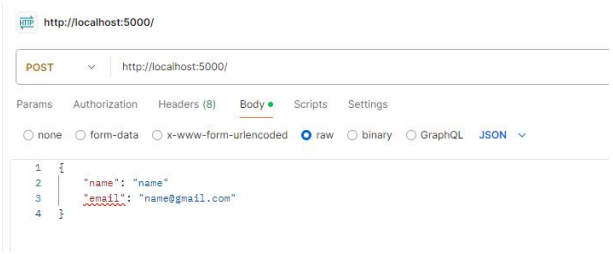
mongoDB.js

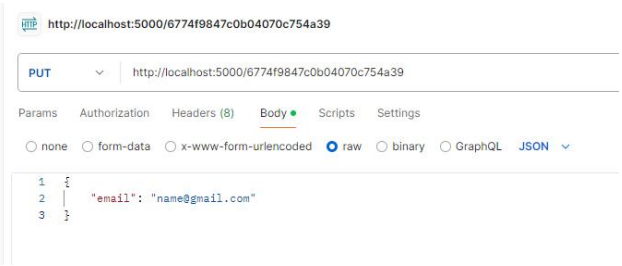
```
const {MongoClient} = require("mongodb");
const mongodburl = "mongodb://localhost:27017/";
const client = new MongoClient(mongodburl);

async function dbconnection (){
    let connection = await client.connect();
    //console.log(connection);
    return connection.db("nodejsDB");
}
//dbconnection();

async function productCollection (){
    let connection = await dbconnection();
    //console.log(connection);
    return connection.collection("collection01");
}
// productCollection();

async function fetchProductData(){
    let collection = await productCollection()
    //console.log(collection)
    let data = await collection.find({}).toArray();
    console.log(data);
}
// fetchProductData();

module.exports = dbconnection
module.exports = productCollection
```

index.js

```
const express = require("express");
const mongodb = require("mongodb"); // use for get ObjectId via params from mongodb collection
const productCollection = require("./mongoDB");
const app = express();

// for check that json data is comming or not from postman
app.use(express.json());
```

```
app.get('/', async (req, resp)=>{
    let data = await productCollection();
    //console.log(data);
    data = await data.find({}).toArray();
    //console.log(data);
    resp.send(data);
})

app.post('/', async (req, resp) => {
    //console.log(req.body);
    let data = await productCollection();
    data = await data.insertOne(req.body);
    //data = req.body;
    resp.send(data);
});

app.put('/:name', async(req,resp)=>{
    let data = await productCollection();
    data = await data.updateOne({ name: req.params.name},{ $set: req.body });
    resp.send({ result: "updated"});
})

app.delete('/:id', async(req,resp)=>{
    let data = await productCollection();
    data = await data.deleteOne({ _id: new mongodb.ObjectId(req.params.id)});
    resp.send({ result: "updated"});
})

app.listen(5000);
```





-------- Mongoose Simple CURD using Node JS with mongoDB and Static data --------

```
const mongoose = require('mongoose');
const express = require("express");
const app = express();

mongoose.connect("mongodb://localhost:27017/nodejsDB");

const productSchema = mongoose.Schema({
    name: String,
    email: String,
});

const save = async () => {
    const productModel = mongoose.model('products', productSchema);
    let data = new productModel({ name: "Raju Roy", email: "raju@gmail.com" });
    let result = await data.save();
    console.log(result);
}

// save();

const update = async () => {
    const productModel = mongoose.model('products', productSchema);
    let data = await productModel.updateOne(
        { name: "rahul" },
        { $set: { name: "Rahul Bose" } }
    );
    console.log(data);
}

// update()

const deleteProduct = async () => {
    const productModel = mongoose.model('products', productSchema);
    let data = await productModel.deleteOne({ name: "Raju Roy" });
    console.log(data);
}
// deleteProduct();

const findProduct = async () => {
    const productModel = mongoose.model('products', productSchema);
    // let data = await productModel.find({ name: "ganesh" });
    let data = await productModel.find();
    console.log(data);
}
// findProduct();
```

Mongoose CURD in Node JS using mongoDB and API Search API

config.js

```
const mongoose = require('mongoose');
mongoose.connect("mongodb://localhost:27017/nodejsDB");
```

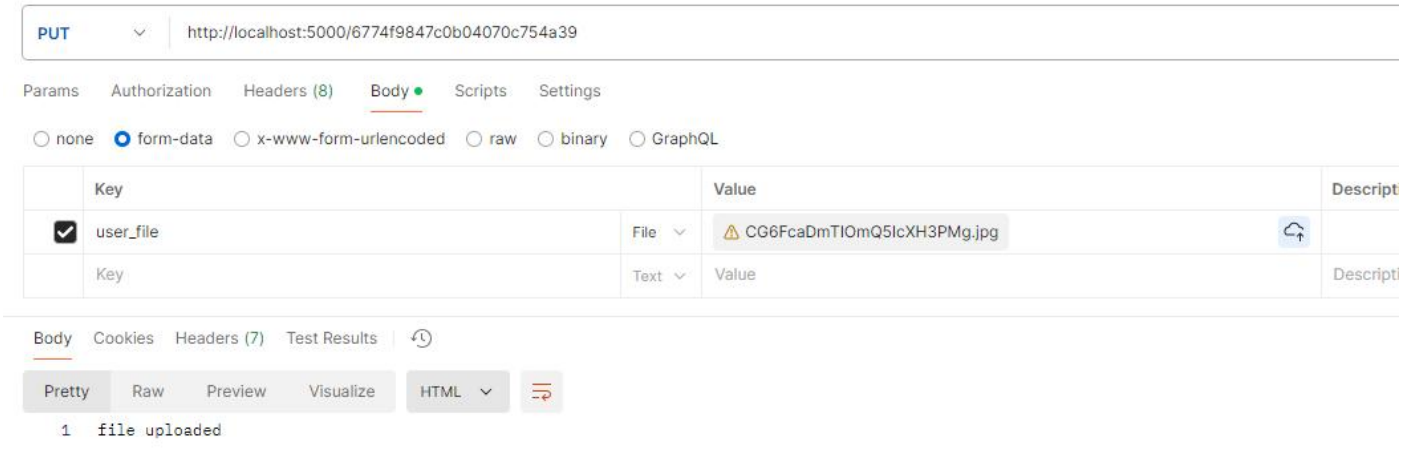
index.js

```
const express = require("express");
require('./config');
const Product = require('./product');
const app = express();

// for check that json data is comming or not from postman
app.use(express.json());

app.post('/create', async (req,resp)=>{
    console.log(req.body)
    let data = new Product(req.body);
    const result = await data.save();
    console.log(result);
    resp.send(result);
});

app.get('/list', async (req,resp)=>{
    let data = await Product.find();
    console.log(data);
    resp.send(data);
});

app.delete('/delete/:_id', async (req,resp)=>{
    let data = await Product.deleteOne(req.params);
    console.log(data);
    resp.send(data);
});

app.put('/update/:_id', async (req,resp)=>{
    let data = await Product.updateOne(req.params, { $set: req.body  });
    console.log(data);
    resp.send(data);
});

app.get('/search/:key', async (req,resp)=>{
        console.log(req.params.key)
        let data = await Product.find({
        "$or": [
                    { "name": {$regex:req.params.key} },
                    { "email": {$regex:req.params.key} }
                ]
        });
        resp.send(data);
});


app.listen(5000);
```

File Upload API using Multer

```
const express = require("express");
const multer = require("multer");
require('./config');
const Product = require('./product');
const app = express();

const upload = multer({
    storage: multer.diskStorage({
        destination: function (req, file, callback){
            callback(null, "uploads")
        },
```

```
            filename: function(req, file, callback){
                callback(null, file.fieldname+"-"+Date.now()+".jpg")
            }
        })
    }).single("user_file");

    app.post('/upload', upload, async (req,resp)=>{
        resp.send("file uploaded")
    });

    app.listen(5000);
```



-------- REST API Development --------

◆ Basic steps ----------

```
const mongoose = require('mongoose');


// step 01 - connect Database
mongoose.connect('mongodb+srv://joydipsarkar01:joydip1234@cluster0.4q84a.mongodb.net/')
.then(()=>console.log("Database connected successfully"))
.catch((error)=> console.log(error) )


// step 02 - create schema
const userSchema = new mongoose.Schema({
    username: String,
    password: String,
    age: Number,
    isActive: Boolean,
    tags: [String],
    createdAt : { type: Date, default : Date.now }
    // default use for if we don't pass any data from frontend the it will pass default data
automatically
});


// step 03 - create user model
const User = mongoose.model('User', userSchema);
// ('User', <--- this is the name of the collection in mongo database


// step 04 - main query function
async function runQueryExample() {
    try{
        // create new user type 01 -------------------------------------
        const newUser = await User.create({
            username: 'sanjoybose',
            password: 'sanjoybose1234',
            age: '32',
            isActive: true,
            tags: ['manager'],
        })
        console.log("Craeted new user", newUser);

        // create new user type 02 -------------------------------------
        let data = new User({ username: 'rahulbose',
            password: 'rahulbose1234',
            age: '23',
            isActive: true,
            tags: ['developer', 'designer'],
        });
        let result = await data.save();
        console.log("Craeted new user", result);


        // get all users
        const allUser = await User.find({});
        console.log("Get all users", allUser);


        // get single user
        const singleUser =  await User.find({ isActive: false });
        console.log(singleUser);


        // get first single data
        const singleFUser =  await User.findOne({ username: 'rajibsarkar' });
        console.log(singleFUser);
```

```
        // get ID from last created data
        const getLastCraetedUserById = await User.findById(newUser._id);
        console.log(getLastCraetedUserById)


        // get selected field not id field
        const selectedFields = await User.find().select('username -_id');
        console.log(selectedFields);


        // get limited user and exclude one user
        const limitedUser = await User.find().limit(5).skip(1).select('username -_id');
        console.log(limitedUser)


        //get sorted user
        const sortedUsers = await User.find().select('username age -_id').sort({ age: -1 });
        // -1 = desecding order && 1 = assending order
        console.log(sortedUsers);


        // count data as the basis of any specific data
        const countDocument = await User.countDocuments({ isActive : true });
        console.log(countDocument);


    }catch(error){
        console.log( 'error --> ', error )
    }finally{
        await mongoose.connection.close()
    }
}

runQueryExample();
```

-------- BOOK Store API --------

Folders and Files:

- **controllers folder**
  - book-controller.js
- **database folder**
  - db.js
- **models folder**
  - book.js
- **routes folder**
  - book-routes.js

.env
package.json
server.js

Step 01: ----------
Install npm and packages

1. npm init -y
2. npm i nodemon --save-dev
3. npm i express mongoose dotenv

Step 02: customize package.json ----------

package.json

```
{
  "name": "book-store-rest-api",
  "version": "1.0.0",
  "description": "",
  "main": "index.js", // ------> change to server.js
  "scripts": {
    "test": "echo \"Error: no test specified\" && exit 1",
    "start": "nodemon server.js" //-------> add this line for npm start
  },
  "keywords": [],
  "author": "",
  "license": "ISC",
  "devDependencies": {
    "nodemon": "^3.1.9"
  },
  "dependencies": {
    "dotenv": "^16.4.7",
    "express": "^4.21.2",
    "mongoose": "^8.9.4"
  }
}
```

Step 03: connect database ----------

- **database folder ---> db.js**

```
const mongoose = require('mongoose');
const connectToDB = async()=>{
    try{
        await
mongoose.connect('mongodb+srv://joydipsarkar01:password@cluster0.wxcxo.mongodb.net/books_DB');
        console.log("mongodb connected successfully")
    }catch(error){
        console.log("Database error", error);
        process.exit(1)
    }
}

module.exports = connectToDB;
```

Step 04: import db file in server.js and setup express ----------

Server.js

```
      const connectToDB = require('./database/db')
      const express = require('express');
      const app = express();

      //connect to our database
      connectToDB();
```

Assign port number in .env

```
PORT= 5000
```

Step 05: create server in server.js ----------

Server.js

```
require('dotenv').config();
const connectToDB = require('./database/db')
const express = require('express');
const app = express();
const PORT = process.env.PORT || 5000;

//connect to our database
connectToDB();

//middleware
app.use(express.json())

app.listen(PORT, ()=>{ console.log(`Server is running on port ${PORT}`) })
```

Step 06: create routes for apis ----------

- **routes folder**
  - **book-routes.js**

book-route.js

First we will create a basic route list for our apis and setup basic router

```
const express = require('express');

// create express router
const router = express.Router();

// list of routes
router.get('/get', );
router.get('/get/:id', );
router.post('/add', );
router.put('/update/:id', );
router.delete('/delete/:id', );

module.exports = router;
```

And import above router in server.js file

server.js

```
require('dotenv').config();
const connectToDB = require('./database/db')
const express = require('express');
const bookRoutes = require('./routes/book-routes');
const app = express();
const PORT = process.env.PORT || 5000;

//connect to our database
connectToDB();

//middleware
app.use(express.json())

// routes
app.use('/api/books', bookRoutes)

app.listen(PORT, ()=>{ console.log(`Server is running on port ${PORT}`) })
```

Step 07: create req and resp arrow functions for routes in book-controller.js ----------

- **controllers folder**
  - book-controller.js

book-controller.js

```
// get all book list
const getAllBooks = async(req, resp) => {}

// get single book
const getSingleBook = async(req, resp) => {}

// add new book
const addNewBook = async(req, resp) => {}

// Update single book
const updateSingleBook = async(req, resp) => {}

// delete book
const deleteBook = async(req, resp) => {}

module.exports = { getAllBooks, getSingleBook, addNewBook, updateSingleBook, deleteBook }
```

Here we create functions and export all to book-router.js

Step 08: complete the all route list in book-routes.js ----------

- **routes folder**
  - book-routes.js

book-route.js

```
const express = require('express');
const { getAllBooks, getSingleBook, addNewBook, updateSingleBook, deleteBook } =
require('../controllers/book-controller');

// create express router
const router = express.Router();

// list of routes
router.get('/get', getAllBooks);
router.get('/get/:id', getSingleBook);
router.post('/add', addNewBook);
router.put('/update/:id', updateSingleBook);
router.delete('/delete/:id', deleteBook);

module.exports = router;
```

Step 09: Create model in book.js ----------

- **models folder**
  - book.js

book.js

```
const mongoose = require('mongoose');

const bookSchema = new mongoose.Schema({
    title : {
        type : String,
        require: [true, "Book Title is required"],
```

```
            trim: true,
            maxLength: [100, "Book Title can not be more than 100 characters"]
        },
        author : {
            type : String,
            require: [true, "Author name is required"],
            trim: true,
        },
        year: {
            type : Number,
            require: [true, "Publication data required"],
            min: [1000, 'Year must be atleast 1000'],
            max: [ new Date().getFullYear(), 'Year can not be future year'],
        },
        createdAt: {
            type : Date,
            default : Date.now,
        }
    });

    module.exports = mongoose.model('Book', bookSchema);
    // 'Book' is the name of collection in mongodb database
```

Now we will make CURD oparation using this model in **book-controller.js** file

Step 10: Create controller in book-controller.js ----------

- **controllers folder**
  book-controller.js

book-controller.js

```
    const book = require('../models/Book');

    // get all book list
    const getAllBooks = async(req, resp) => {
        try{
            const allBook = await book.find({});
            if(allBook.length > 0){
                resp.status(200).json({
                    success: true,
                    message: "List of books fetch successfully",
                    data : allBook,
                });
            }else{
                resp.status(404).json({
                    status: false,
                    message: "NO books found !!!!"
                });
            }
        }catch(e){
            console.log(e);
            resp.status(500).json({
                success: false,
                message: "Something went wrong !!!"
            });
        }
    }

    // get single book
    const getSingleBook = async(req, resp) => {
        try{
            const singleBook = await book.findById(req.params.id);
            if(!singleBook){
                resp.status(404).json({
                    status: false,
                    message: "NO books found !!!!"
                });
            }
```

```
            resp.status(200).json({
                success: true,
                message: "book fetch successfully",
                data : singleBook,
            });
        }catch(e){
            console.log(e);
            resp.status(500).json({
                success: false,
                message: "Something went wrong !!!"
            });
        }
    }

    // add new book
    const addNewBook = async(req, resp) => {
        try{
            const newBookFormData = req.body;
            const newlyCreatedBook = await book.create(newBookFormData);
            if(newlyCreatedBook){
                resp.status(201).json({
                    success: true,
                    message: "New Book added succussfully",
                    data: newlyCreatedBook
                })
            }
        }catch(error){
            console.log(error);
            resp.status(500).json({
                success: false,
                message: "Something went wrong !!!"
            });
        }
    }

    // Update single book
    const updateSingleBook = async(req, resp) => {
        try{
           const getUpdatedFormData = req.body;
           const getCurrentBookID = req.params.id;
           const updateBook = await book.findByIdAndUpdate(
            getCurrentBookID,
            getUpdatedFormData,
            {
                new: true
            });
            if(!updateBook){
                resp.status(404).json({
                    status: false,
                    message: "NO books found !!!!"
                });
            }
            resp.status(200).json({
                success: true,
                message: "book update successfully",
                data : updateBook,
            });
        }catch(error){
            console.log(error);
            resp.status(500).json({
                success: false,
                message: "Something went wrong !!!"
            });
        }
    }

    // delete book
    const deleteBook = async(req, resp) => {
        try{
            const getBookID = req.params.id;
            const deletedBook = await book.findByIdAndDelete(getBookID);
            if(!deletedBook){
                resp.status(404).json({
```

```
                    status: false,
                    message: "NO books found !!!!"
                });
            }
            resp.status(200).json({
                success: true,
                message: "book deleted successfully",
                data : deletedBook,
            });
        }catch(error){
            console.log(error);
            resp.status(500).json({
                success: false,
                message: "Something went wrong !!!"
            });
        }
    }

    module.exports = { getAllBooks, getSingleBook, addNewBook, updateSingleBook, deleteBook }
```

-------- Authentication and authorization --------

Folders and Files:

- controllers folder
  - auth-controller.js
- database folder
  - db.js
- middleware folder
  - admin-middleware.js
  - auth-middleware.js
- models folder
  - User.js
- routes folder
  - admin-routes.js
  - auth-routes.js
  - home-routes.js

.env
package.json
server.js

Encrypted/hash Password: npm i bcrypt
Json Web Token : npm i jsonwebtoken

db.js

```
const mongoose = require('mongoose');
const connectDB = async () => {
    try{
        await
mongoose.connect("mongodb+srv://joydipsarkar01:joydip1234@cluster0.wxcxo.mongodb.net/books_DB");
        console.log("MongoDB connected");
    }catch(error){
        console.log(error, "Database connection error !!!!")
    }
}
module.exports = connectDB;
```

server.js

```
require('dotenv').config();
const PORT = process.env.PORT;
const connectDB = require('./database/db');
const express = require('express');
const authRouters = require('./routers/auth-routes');
const homeRouters = require('./routers/home-routes');
const adminRouters = require('./routers/admin-routes');
const app = express();

// database connection
connectDB();

// middleware
app.use(express.json());

//routers
app.use('/api/auth', authRouters)
app.use('/api/home', homeRouters)
app.use('/api/admin', adminRouters)

// server listen
app.listen(PORT, ()=>{
    console.log(`Server is runing on port ${PORT}`);
});
```

.env

```
PORT = 5000
JWT_SECRETE_KEY = JWT_SECRETE_KEY
```

- **models folder**
  - User.js

User.js

```
const mongoose = require('mongoose');

const authSchema = mongoose.Schema({
    username: {
        type: String,
        required: true,
        unique: true,
        trim: true,
    },
    email: {
        type: String,
        required: true,
        unique: true,
        trim: true,
        lowercase: true,
    },
    password:{
        type: String,
        required: true,
    },
    role:{
        type: String,
        enum: ['user', 'admin'], // only allows 'user' or 'admin' roles
        default: 'user'
    }
}, { timestamps :true });

module.exports = mongoose.model('User', authSchema);
```

auth-routes.js

```
const express = require('express');
const { authLogin, authRegister } = require('../controllers/auth-controllers');

// create routes
const router = express.Router();

// list of routes
router.post('/register', authRegister);
router.post('/login', authLogin);

module.exports = router;
```

auth-controller.js

```
const bcrypt = require('bcrypt');
const jwt = require('jsonwebtoken');
const User = require('../models/User');

// register
const authRegister = async (req, resp) => {
    try{
        // extract user information from request body
        const { username, email, password, role } = req.body;
```

```
        //if check the user is already exits in our database
        const checkExitingUser = await User.findOne({
            $or: [{ username}, {email}]
        });

        if(checkExitingUser){
            resp.status(400).json({
                success: false,
                message: "User already exist !!!!!"
            });
        }

        // hash password for encrypted the password
        const salt = await bcrypt.genSalt(10);
        const hashedPassword = await bcrypt.hash(password, salt);

        // create a user in database
        const newlyCreatedUser = new User({
            username,
            email,
            password: hashedPassword,
            role : role || 'user'
        });

        console.log("newdata  ===== ", newlyCreatedUser)

        await newlyCreatedUser.save();

        if(newlyCreatedUser){
            resp.status(200).json({
                success: true,
                message: "New User added",
                data: newlyCreatedUser
            });
        }else{
            resp.status(400).json({
                success: false,
                message: "Unable to register the user"
            });
        }

    }catch(error){
        resp.status(404).json({
            success: false,
            message : "Some error occured !!!"
            }
        )
    }
}


// login
const authLogin = async (req, resp) => {
    try{
        // extract user information from request body
        const { username, password } = req.body;

        // find the current user is exist in database
        const user = await User.findOne({username});

        if(!user){
            resp.status(400).json({
                success: false,
                message: "invalid credencials"
            });
        }

        // check the password is match or not
        const passwordMatch = await bcrypt.compare(password, user.password);

        if(!passwordMatch){
            resp.status(400).json({
                success: false,
```

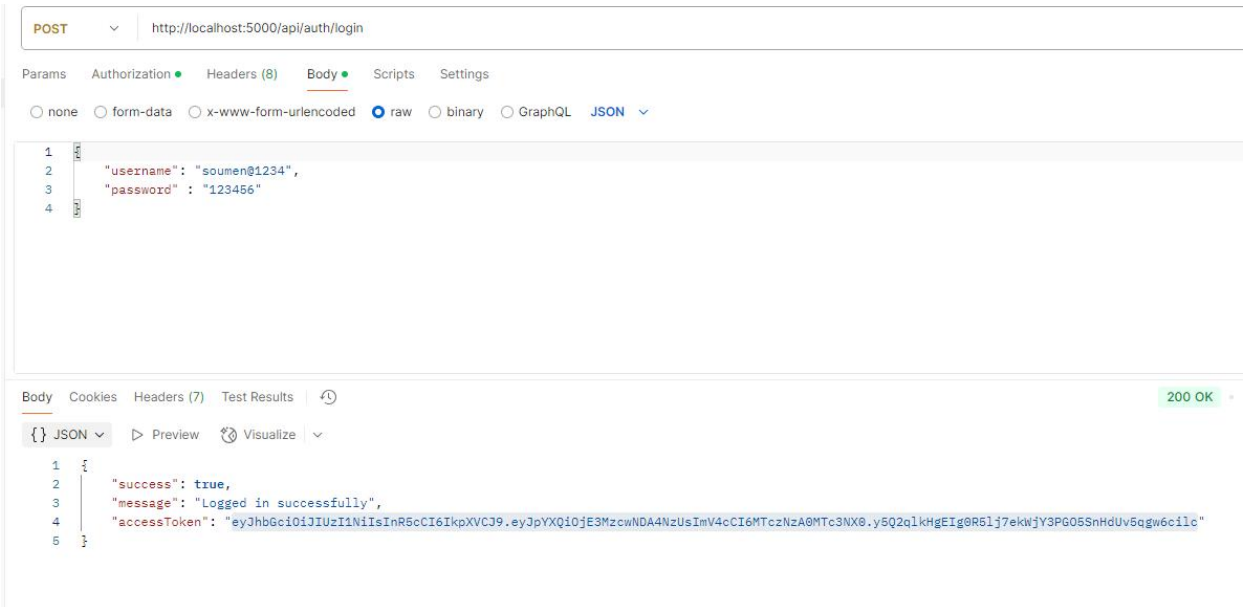
```
                message: "invalid credencials"
            });
        }

        // create user token
        const accessToken = jwt.sign({
            id: user._id,
            username: user.username,
            role: user.role
        }, process.env.JWT_SECRETE_KEY, {
            expiresIn: '15m'
        });

        resp.status(200).json({
            success: true,
            message: "Logged in successfully",
            accessToken
        });

    }catch(error){
        resp.status(500).json({
            success: false,
            message : "Some error occured !!!"
            }
        )
    }
}

module.exports =  { authLogin, authRegister }
```

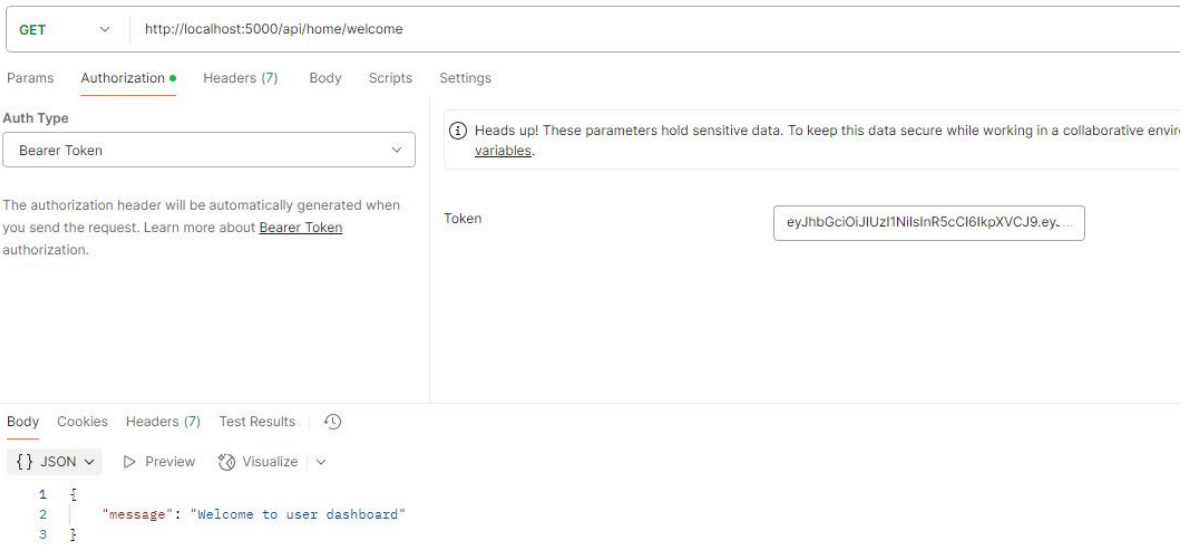
auth-middleware.js

```
const jwt = require('jsonwebtoken');
require('dotenv').config();

const authMiddleware = (req, resp, next) =>{
    console.log("middleware running");

    const authHeader = req.headers['authorization'];
    console.log(authHeader);

    const token = authHeader && authHeader.split(" ")[1];

    if(!token){
        return resp.status(401).json({
            success: false,
            message: "No token provided"
        })
    }

    // decode token
    try{
        const decordedTokenInfo = jwt.verify(token, process.env.JWT_SECRETE_KEY);
        console.log(decordedTokenInfo);
        req.userInfo = decordedTokenInfo;
        next();

    }catch(error){
        return resp.status(500).json({
            status: false,
            message: "No token provided"
        })
    }

}
module.exports = authMiddleware;
```

home-routes.js

```
const express = require('express');
const authMiddleware = require('../middleware/auth-middleware');
const router = express.Router();

router.get('/welcome', authMiddleware, (req, resp)=>{

    const { id, username, role } = req.userInfo;
    resp.json({
        message: "Welcome to user dashboard",
        user: {
            _id: id,
            username,
            role
        }
    })
})

module.exports = router;
```



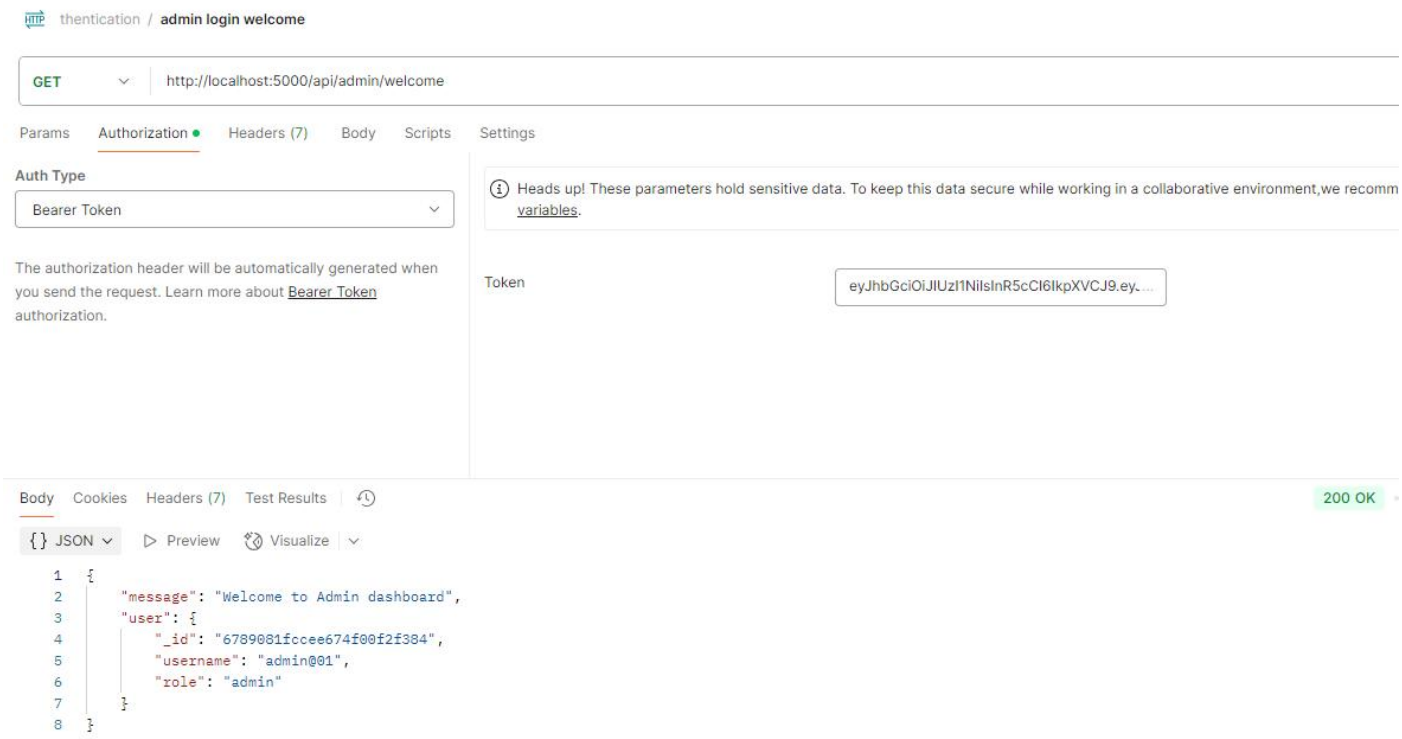


admin-middlewares.js

```
const adminMiddleware = (req, resp, next) => {
    if(req.userInfo.role !== "admin"){
        return resp.status(403).json({
            success: false,
            message: "Access Denied! Admin can access only"
        })
    }

    next();
}

module.exports = adminMiddleware;
```

admin-routes.js

```
const express = require('express');
const authMiddleware = require('../middleware/auth-middleware');
const adminMiddleware = require('../middleware/admin-middleware');
const router = express.Router();

router.get('/welcome', authMiddleware, adminMiddleware, (req, resp)=>{
    const { id, username, role } = req.userInfo;
    resp.json({
        message: "Welcome to Admin dashboard",
        user: {
            _id: id,
            username,
            role
        }
    })
})

module.exports = router;
```


thentication / admin login welcome
GET
http://localhost:5000/api/admin/welcome
Params
Authorization
Headers (7)
Body
Scripts
Settings
Auth Type
Bearer Token
The authorization header will be automatically generated when you send the request. Learn more about Bearer Token authorization.
Heads up! These parameters hold sensitive data. To keep this data secure while working in a collaborative environment,we recomm
variables.
Token
eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.ey...
Body
Cookies
Headers (7)
Test Results
200 OK
{} JSON
Preview
Visualize
{
"message": "Welcome to Admin dashboard",
"user": {
"_id": "6789081fccee674f00f2f384",
"username": "admin@01",
"role": "admin"
}
}

onwebtok